

Deterministic Gambit

1. Project Overview

This project aims to develop a playable chess-based strategy game that introduces a modified pawn promotion rule while incorporating an AI opponent. The game will follow standard chess mechanics with one major change: pawn promotion will depend on the file (column) where the pawn started.

2. Project Review

A relevant existing project is the chess engine Stockfish, one of the strongest open-source chess engines available. Stockfish calculates optimal moves using search algorithms and advanced board evaluation techniques. While Stockfish provides powerful move calculation capabilities, it does not provide a graphical user interface or built-in gameplay statistics tracking. Additionally, it follows the standard chess rules and does not implement alternative promotion systems.

This project builds upon the capabilities of Stockfish by:

- Creating a **custom chess game interface**
- Introducing a **file-based pawn promotion variant**
- Integrating Stockfish as an **AI opponent**
- Recording **gameplay statistics for analysis**

3. Programming Development

3.1 Game Concept

The game is a chess-based strategy game played between a human player and an AI opponent. The objective of the game remains the same as traditional chess: to checkmate the opponent's king. However, the game introduces a modified pawn promotion rule. When a pawn reaches the last

rank, the piece it promotes to is determined by the file where the pawn started. This rule changes pawn strategy significantly. Players must consider pawn positioning carefully since the final promotion piece is predetermined.

Key features of the game include:

- Human vs AI gameplay
- custom pawn promotion system
- full chess rule implementation
- move validation
- gameplay statistics collection
- statistical data visualization

3.2 Object-Oriented Programming Implementation

1. Game Class

Role:

Manages the overall game flow.

Attributes:

board - stores the current Board object

current_player - indicates whose turn it is (white or black)

game_over - a boolean flag to indicate if the game has ended

move_history - list of all moves played in the game

Methods:

start_game() - Initializes the game and starts the main loop

switch_turn() - Alternates the current player after each valid move

check_game_end() - Determines if the game has ended (checkmate, stalemate, or draw)

2. Board Class

Role:

Represents the chessboard and maintains the positions of all pieces.

Attributes:

board_grid - a 2D array (8×8) storing piece objects or empty spaces

piece_list - a list of all active pieces on the board

Methods:

initialize_board() - Places all pieces in their starting positions

move_piece() - Moves a piece from one position to another and updates the board state

get_piece() - Returns the piece at a specific position

3. Piece Class

Role:

Parent class for all chess pieces.

Attributes:

color - the piece's color (white or black)

position - current location on the board

piece_type - type of piece

Methods:

get_valid_moves() - Returns all legal moves based on the current board state

Subclasses:

Pawn

Rook

Knight

Bishop

Queen

King

4. AIEngine Class

Role:

Handles communication with the AI engine.

Attributes:

engine_path - file path to the Stockfish executable

Methods:

get_best_move() - Sends the current board state to the engine and retrieves the best move

send_position() - Converts the board into a format that the engine can understand

receive_move() - Receives and parses the move returned by the engine

5. StatisticsManager Class

Role:

Records and stores gameplay statistics.

Attributes:

game_duration - Stores the total time taken to complete a game

move_count - Tracks the total number of moves made during the game

promotion_count - Counts how many pawn promotions occur in a game

promotion_types - A list storing the types of pieces obtained from promotions

game_results - Stores the final outcome of the game (Player Win / AI Win / Draw)

evaluation_history - A list that records the evaluation score at each turn, used for time-series analysis

Methods:

record_move() - Updates the move count and appends the current evaluation score to evaluation_history

record_promotion() - Increments the promotion count and stores the promotion type in promotion_types

save_game_data() - Stores the final result and total duration once the game is finished

3.3 Algorithms Involved

Rule-Based Movement Algorithms: Each chess piece follows movement rules that determine valid moves.

Move Validation Algorithm: The system checks whether a move is legal before executing it, ensuring that moves follow chess rules.

Game State Detection: Algorithms will detect conditions such as checkmate and stalemate.

File-Based Pawn Promotion Algorithm: The project implements a custom promotion system where the promotion result is determined by the pawn's starting file (column). When a pawn reaches the final rank, its original file is checked and mapped to a specific piece type (e.g., rook, knight, bishop, or queen). This algorithm introduces strategic constraints and differentiates the game from standard chess.

AI Move Search: The AI opponent will generate moves using the chess engine Stockfish, which applies search algorithms and board evaluation functions to determine strong moves.

4. Statistical Data (Prop Stats)

4.1 Data Features

Feature Name	Why it is good to have this data? What can it be used for	How will you obtain 100 values of this feature data?	Which variable (and which class will you collect this from)?	How will you display this feature data (via summarization statistics or via graph)?
Game Duration	Helps analyze how long games typically last and compare efficiency between wins/losses	Record duration for each completed game (100 games)	game_duration from StatisticsManager	Scatter plot, mean, median, max, min, SD
Total Moves	Shows how long games are in terms of complexity and depth	Count moves for each game played	move_count from StatisticsManager	Histogram, scatter plot, mean, median, max, min, SD
Promotion Count	Measures how often special pawn events occur	Increment whenever a promotion happens	promotion_count from StatisticsManager	Bar chart, mean, median, max, min, SD
Promotion Types	Shows which pieces are most commonly promoted to	Record the type each time a promotion occurs	promotion_types from StatisticsManager	Bar chart
Game Result	Used to evaluate performance (win/loss/draw distribution)	Record the result after each game ends	game_result from StatisticsManager	Pie chart

Evaluation Score	Tracks the advantage changes during a game	Record score after every move	evaluation_history from StatisticsManager	Time-series line graph
------------------	--	-------------------------------	---	------------------------

4.2 Data Recording Method

Gameplay data will be saved in a CSV file after each completed game by my friends in the chess club, then combine each CSV file together to get 100 records.

Example format:

Game-Level Data:

Duration	Moves	Promotions	PromotionTypes	Result
8.4	40	2	"rook,queen"	Player Win

Turn-Level Data:

Turn	Evaluation Score
3	+0.23

4.3 Data Analysis Report

Table:

Metric	Moves	Duration	Promotions
Mean	Mean of moves	Mean of duration	Mean of promotions
Median	Median of moves	Median of duration	Median of promotions
Max	Max of moves	Max of duration	Max of promotions
Min	Min of moves	Min of duration	Min of promotions

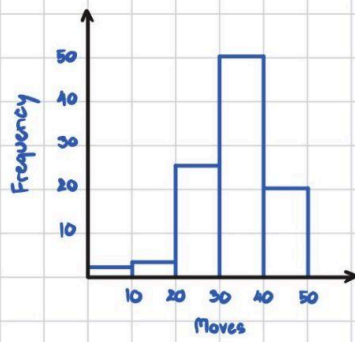
Standard Deviation	SD of moves	SD of duration	SD of promotions
--------------------	-------------	----------------	------------------

Graphs:

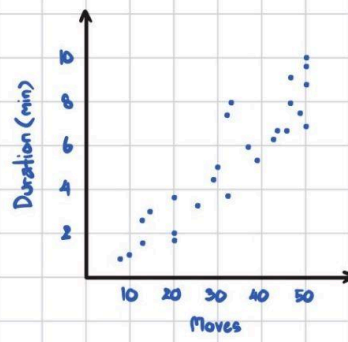
	Feature Name	Graph Objective	Graph Type	X-axis	Y-axis
Graph 1	Total Moves	Analyze the distribution of game lengths	Histogram	Moves	Frequency
Graph 2	Moves vs Duration	Examine the relationship between moves and time	Scatter Plot	Moves	Duration
Graph 3	Game Result	Show proportion of outcomes (win/loss/draw)	Pie Chart	-	-
Graph 4	Promotion Types	Compare the frequency of each promotion type	Bar Chart	Promotion Type	Count
Graph 5	Evaluation Score	Analyze how the advantage changes over time	Line Graph	Turn Number	Evaluation Score

Hand-drawn expected graphs:

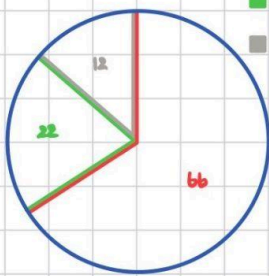
Total Moves



Moves and Duration

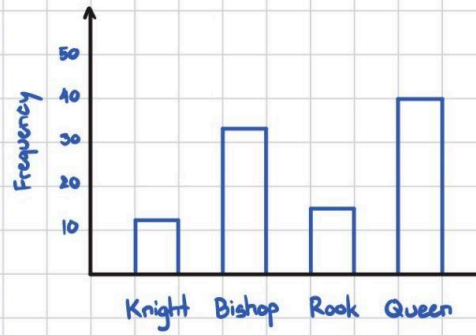


Game Result

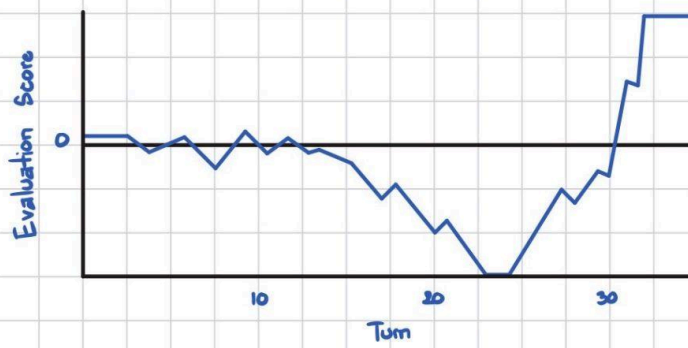


- Losses
- Wins
- Draws

Promotion Types



Evaluation Score (Last Game)



5. Project Planning Timeline

Week	Week	Task
1	8	Proposal submission / Project initiation
2	9	Implement the Board class and base Piece class
3	10	Implement piece movement rules and file-based pawn promotion logic
4	11	Implement the Game class to manage turns, move execution, and integrate the AI opponent
5	12	Implement the StatisticsManager class and develop the data recording system
6	13	Develop data analysis and visualization, and collect at least 100 gameplay records
7	14	Debugging, optimizing, finalizing UI and statistical outputs, and submission

6. Document version

Version: 1.0

Date: 4 March 2025

Editor: Kantinan Lertluckpreecha

Version: 2.0

Date: 28 March 2025

Editor: Kantinan Lertluckpreecha

Version: 2.1

Date: 30 March 2025

Editor: Kantinan Lertluckpreecha

Version: 2.2

Date: 31 March 2025

Editor: Kantinan Lertluckpreecha

Date	Name	Feedback
14/03	Peraya	Overall Feedback: Your game concept is good, and your OOP implementation (Section 3.2) is quite okay, particularly the use of inheritance for the Piece class. However, there is a critical flaw in your data collection strategy that will make it practically impossible to meet the grading requirements without hundreds of hours of playtesting. Major Changes Needed - 4.1 & 4.2 Data Features and Recording (Data Volume Risk) - Comment: Your current CSV structure logs data <i>per game</i> (1 row = 1 game). To meet the “minimum 100 records per feature” requirement, you would have to play 100 full games of chess against Stockfish. This is completely unfeasible. Furthermore, if multiple pawns are promoted in a single game, logging them in a single PromotionType column will break your CSV structure. - You must change your logging frequency from <i>per game</i> to <i>per move</i> (or per turn). If a standard chess game takes 40 moves, logging per move guarantees you hit 100+ records in just 2-3 games. - Suggested Feature Replacements (Per Move): Track features like Turn Number, Piece Moved, Move Duration (seconds spent thinking), Stockfish Eval Score (advantage calculation), and Is Promotion (True/False/Piece Type). - 4.3 Data Analysis Report (Missing Specifics) - You listed generic chart types (bar charts, pie charts), but you did not link them to the data you are collecting. - Action Required: Explicitly state what each chart will display. For example: A Pie Chart displaying the distribution of Game Results (Win/Loss/Draw) or A Histogram showing the frequency of total moves per game. Minor Recommendations

		- 3.3 Algorithms Involved: You mention Stockfish applies search algorithms. While true, since Stockfish is an external engine, make sure your actual Python code includes algorithmic logic of its own, such as your custom File-Based Promotion Logic. Ensure that logic is explicitly mentioned here so you get credit for writing algorithms, rather than just calling an API or external code. - Proofreading & Grammar: Please review the document to correct minor English grammar and wording issues. A technical proposal should be professionally written and easy to read.
15/03	Amornrit	Sounds good 👍 Not overly complex nor too simple. But I have a concern about your data collection; logging one game per row is very tedious, as 100 records is a minimum requirement. I suggest you track other features to save your time. If you still want to keep those features, you might consider using automation. Data analysis needs more explanation and detail. Minor issue, section 3.2 should be more structured for better readability.
30/03	Peraya	Despite previous feedback from us, your proposal still states in Section 4.1: Record duration for each completed game While you mentioned having friends in the chess club help, this still requires an unrealistic amount of manual effort in a limited time (playing 100 full chess games). You also have a Turn-Level Data in section 4.2, which is good. However, your core requirements in the table and your Graph Plan are still almost entirely dependent on Game-Level metrics. To get 100 data points you still need 100 games.